Group SHAP for Correlated Features — A Practical Guide

Why feature-level SHAP misleads on multicollinear data, and how to fix it with grouped attributions.

Audience: ML practitioners building tabular models where features come in natural clusters (transaction velocity windows, multi-bureau scores, device signals, time-window aggregates, etc.) and where those clusters are highly correlated.

Stack used: Python 3.10+, `shap>=0.44`, `lightgbm`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn`.

---

1. The problem in one paragraph

Vanilla SHAP assigns contribution to individual features. When two features carry nearly the same signal (Pearson r ≥ 0.7), the tree ensemble splits credit between them essentially at random — one feature appears important, the other appears near-zero, and the assignment can flip between retrains, between folds, or even between explainer backgrounds. This is not a bug. It is a direct consequence of Shapley fairness: if `X1` and `X2` are interchangeable, the axiom of symmetry guarantees only that `φ(X1) + φ(X2)` is stable — the individual split is not. Interpreting either feature alone is therefore unreliable. The solution is to stop pretending they are separate players in the coalition and treat the correlated set as one.

2. Why grouping is the right fix

Shapley's formalism allows a "player" to be a single feature *or a coalition of features*. If you group `{X1, X2}` and treat them as a single player, the attribution to that player is:

$$\phi(\text{group}) = \phi(X_1) + \phi(X_2)$$

This is provable from the efficiency and symmetry axioms — **summing the raw (signed) SHAP values inside a group is a legitimate Shapley value for the group as a player.** You are not approximating anything. The loss of information is exactly the loss you wanted: the unstable allocation *within* the group is collapsed; the stable total contribution *of* the group is preserved.

Two critical caveats:

1. **Sum raw, not absolute, values.** Absolutes destroy cancellation effects. If X1 pushes risk up and X2 pushes it down on the same row, the group's true contribution is the net, not the sum of magnitudes.
2. **Groups must be defined before SHAP is computed.** Post-hoc grouping based on what you see in the SHAP plot is cherry-picking and will not survive a validation review.

3. Setup

```
import numpy as np

import pandas as pd

import shap

import lightgbm as lgb

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

shap.initjs()

rng = np.random.default_rng(42)
```

4. A realistic synthetic dataset

We build a fintech-flavoured fraud/default dataset with known correlation structure so the "right answer" is verifiable.

```
n = 5000

# --- Group A: transaction velocity features (highly correlated, r ~ 0.9) ---

latent_velocity = rng.normal(0, 1, n)

tx_vel_1h  = latent_velocity + rng.normal(0, 0.3, n)

tx_vel_24h = latent_velocity + rng.normal(0, 0.3, n)

tx_vel_7d  = latent_velocity + rng.normal(0, 0.3, n)
```

```
# --- Group B: credit bureau scores (correlated, r ~ 0.85) ---

latent_bureau = rng.normal(0, 1, n)

bureau_cibil    = latent_bureau + rng.normal(0, 0.4, n)

bureau_experian = latent_bureau + rng.normal(0, 0.4, n)

# --- Group C: device risk (negatively correlated within group) ---

latent_device = rng.normal(0, 1, n)

device_age_days    = latent_device + rng.normal(0, 0.4, n)

device_trust_score = -latent_device + rng.normal(0, 0.4, n)

# --- Independent features ---

income_log = rng.normal(0, 1, n)

age_std    = rng.normal(0, 1, n)

# --- Target: driven by the latents, not the noisy observed features ---

logit = (0.9 * latent_velocity

         - 0.8 * latent_bureau

         + 0.6 * latent_device

         + 0.4 * income_log

         + rng.normal(0, 0.5, n))

y = (logit > np.quantile(logit, 0.7)).astype(int)   # ~30% positive rate

X = pd.DataFrame({

    "tx_vel_1h": tx_vel_1h,

    "tx_vel_24h": tx_vel_24h,

    "tx_vel_7d": tx_vel_7d,
```

```
    "bureau_cibil": bureau_cibil,
    "bureau_experian": bureau_experian,
    "device_age_days": device_age_days,
    "device_trust_score": device_trust_score,
    "income_log": income_log,
    "age_std": age_std,
})
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

Quick sanity check — confirm the correlation structure is what we designed:

```
plt.figure(figsize=(8, 6))
sns.heatmap(X_train.corr().abs(), annot=True, fmt=".2f",
            cmap="viridis", vmin=0, vmax=1, square=True)
plt.title("Absolute Pearson correlation — raw features")
plt.tight_layout()
plt.show()
```

What to look for: three high-correlation blocks on the diagonal — velocity (3×3, values ≈ 0.9), bureau (2×2, ≈ 0.85), device (2×2, ≈ 0.85 in absolute terms; `device_trust_score` will show negative correlation before we take the abs). `income_log` and `age_std` should be roughly zero-correlated with everything.

5. Train the model

```
model = lgb.LGBMClassifier(
```

```
    n_estimators=400,
    learning_rate=0.05,
    num_leaves=31,
    min_child_samples=30,
    random_state=42,
    verbose=-1,
)
model.fit(X_train, y_train)
print(f"Train AUC: {model.score(X_train, y_train):.3f}")
print(f"Test AUC:  {model.score(X_test, y_test):.3f}")
```

6. Compute SHAP values and see the problem

```
explainer = shap.TreeExplainer(model)
sv = explainer.shap_values(X_test)
# LightGBM binary: shap versions differ. Normalise to (n_samples, n_features) for class 1.
if isinstance(sv, list):
    sv = sv[1]
elif sv.ndim == 3:
    sv = sv[:, :, 1]
sv_df = pd.DataFrame(sv, columns=X_test.columns, index=X_test.index)
shap.summary_plot(sv, X_test, plot_type="bar", show=True)
shap.summary_plot(sv, X_test, show=True)   # beeswarm
```

What you should see — and why it is a problem

- **Bar chart:** one velocity feature will dominate (say `tx_vel_24h` at the top), the other two will appear middling. Intuitively, all three windows carry the *same* information — yet the bar chart implies one is much more important. This is exactly the arbitrary credit-splitting behaviour. Retrain with a different seed and a different velocity feature will "win".
- **Beeswarm:** for the two bureau features, you will often see `bureau_cibil` with wide SHAP dispersion and `bureau_experian` compressed near zero — or vice versa. The model is using one as the primary split and treating the other as a redundant near-duplicate.

Validator takeaway: if a stakeholder reads this plot and concludes "`bureau_experian` is unimportant, we can drop it" — they are wrong. Drop it and `bureau_cibil` will absorb its contribution in the next retrain. The joint signal is what matters.

Confirming the instability — run SHAP across bootstrap refits

```
def feature_rank(X_tr, y_tr, X_te, seed):

    m = lgb.LGBMClassifier(n_estimators=400, learning_rate=0.05, num_leaves=31,

                           min_child_samples=30, random_state=seed, verbose=-1)

    m.fit(X_tr, y_tr)

    s = shap.TreeExplainer(m).shap_values(X_te)

    if isinstance(s, list): s = s[1]

    elif s.ndim == 3: s = s[:, :, 1]

    return pd.Series(np.abs(s).mean(axis=0), index=X_te.columns).rank(ascending=False)

ranks = pd.DataFrame({seed: feature_rank(X_train, y_train, X_test, seed)

                      for seed in range(10)})

print("Rank standard deviation across 10 seeds:")

print(ranks.std(axis=1).sort_values(ascending=False))
```

What to look for: the three velocity features and the two bureau features will have rank std > 1 (often 2–3). Truly independent features (`income_log`, `age_std`) will have std near zero. This is your empirical proof, for the audit trail, that feature-level SHAP is unstable on this data.

7. Compute Group SHAP

Step 7.1 — Define groups before looking at SHAP output

```
groups = {

    "transaction_velocity": ["tx_vel_1h", "tx_vel_24h", "tx_vel_7d"],

    "bureau_score":         ["bureau_cibil", "bureau_experian"],

    "device_risk":          ["device_age_days", "device_trust_score"],

    "income_log":           ["income_log"],

    "age_std":              ["age_std"],

}
```

Single-member groups are kept intentionally — they carry through untouched and the total-contribution arithmetic still works.

Step 7.2 — Sum raw SHAP values within each group

```
def group_shap(sv_df: pd.DataFrame, groups: dict) -> pd.DataFrame:

    return pd.DataFrame(

        {g: sv_df[cols].sum(axis=1) for g, cols in groups.items()},

        index=sv_df.index,

    )

group_sv = group_shap(sv_df, groups)

print(group_sv.describe())
```

Step 7.3 — The non-negotiable sanity check

Total group SHAP across all groups, plus the expected value, must equal the model's raw-score prediction for every row. If it doesn't, your grouping is wrong (usually: a feature is in two groups, or missing).

```
expected_value = explainer.expected_value

if hasattr(expected_value, "__len__"):

    expected_value = np.asarray(expected_value).flatten()[-1]

reconstructed   = group_sv.sum(axis=1) + expected_value

model_raw_score = model.predict(X_test, raw_score=True)

assert np.allclose(reconstructed.values, model_raw_score, atol=1e-5), \

    "Group SHAP does not reconstruct model output — check group definitions."

print("✓ Group SHAP reconstructs model logits exactly.")
```

Make this an assertion in the validation pipeline. It catches grouping bugs that no amount of plot-gazing will.

8. Interpreting the output

8.1 Global importance — mean |group SHAP|

```
importance = group_sv.abs().mean().sort_values(ascending=True)

plt.figure(figsize=(8, 5))

importance.plot(kind="barh", color="#2E86AB")

plt.xlabel("mean(|group SHAP|)")

plt.title("Group-level global importance")

plt.tight_layout()

plt.show()
```

What to look for: a stable ordering that matches the known generative weights (velocity ≈ 0.9, bureau ≈ 0.8, device ≈ 0.6, income ≈ 0.4, age ≈ 0). Compare to the feature-level bar chart from Section 6 — the group version will be dramatically more interpretable and will not shift across retrains. This is the plot you show to stakeholders.

8.2 Group-level beeswarm — distributional view

```
group_expl = shap.Explanation(

    values=group_sv.values,

    base_values=np.full(len(group_sv), expected_value),

    data=group_sv.values,       # no single underlying value for a group; use the SHAP itself for colouring

    feature_names=list(groups.keys()),

)

shap.plots.beeswarm(group_expl, max_display=10, show=True)
```

What to look for:

- **Spread of the dots** — how much a group's contribution varies across the test set. A tight cluster at zero means the group barely moves predictions; a long tail means it's a strong driver for specific subpopulations.
- **Sign balance** — are most contributions positive, negative, or symmetric? For a fraud model, you want the risk-indicator groups (velocity, device) to be asymmetric to the positive (risk-increasing) side and protective groups (bureau_score) to be asymmetric to the negative.
- **Outliers** — a group with a handful of dots far from the main mass points to a subpopulation worth investigating (possible segment, possible data quality issue).

Note: for groups, the colour mapping that beeswarm normally uses (feature value) is not meaningful — a group doesn't have a single value. Either suppress it as above or colour by the group sum itself to show magnitude.

8.3 Intra-group SHAP correlation — the diagnostic plot

This is the plot most validators don't know about and that most clearly demonstrates why grouping was necessary.

```
shap_corr = sv_df.corr(method="spearman")

plt.figure(figsize=(9, 7))

sns.heatmap(shap_corr, annot=True, fmt=".2f",

        cmap="coolwarm", vmin=-1, vmax=1, square=True,

        cbar_kws={"label": "Spearman ρ of raw SHAP values"})

plt.title("SHAP-value correlation across features")

plt.tight_layout()

plt.show()
```

What to look for:

- **Strong positive SHAP correlation (ρ > 0.5) between features in the same group** — confirms they are acting as substitutes for each other. This is the empirical signature of the credit-splitting problem. It validates your grouping.
- **Strong negative SHAP correlation** — features pushing the prediction in opposite directions for the same rows. Inside a group (like our `device_age_days` vs `device_trust_score`) this is expected and is exactly why you sum the *signed* values.
- **Unexpected cross-group correlations** — e.g., a velocity feature's SHAP strongly correlated with a bureau feature's SHAP. This suggests your groups are too narrow; you may have missed a latent common driver. Worth investigating before the validation review.

This is different from, and more informative than, the raw-feature correlation heatmap. Raw-feature correlation tells you the inputs are related. SHAP-value correlation tells you the **model is treating them as related** — which is the property that actually matters for interpretation.

8.4 Dependence view — group SHAP vs. a chosen anchor

For a group, pick the most-variant member as the anchor and plot group SHAP against it:

```
anchor = "tx_vel_24h"

group_name = "transaction_velocity"

plt.figure(figsize=(7, 5))
```

```
plt.scatter(X_test[anchor], group_sv[group_name], s=8, alpha=0.5, c="#2E86AB")

plt.axhline(0, color="grey", lw=0.8)

plt.xlabel(f"{anchor} (anchor feature value)")

plt.ylabel(f"{group_name} — group SHAP")

plt.title(f"Group SHAP dependence on {anchor}")

plt.tight_layout()

plt.show()
```

What to look for: a monotone (or at least coherent) relationship. Non-monotonicity or a flat cloud means either the anchor is not representative of the group, or there's an interaction that deserves explicit investigation (consider SHAP interaction values, `explainer.shap_interaction_values`, scoped to the anchor).

9. Auto-grouping with hierarchical clustering

When you have hundreds of features and defining groups by hand is infeasible, use `shap.utils.hclust` to cluster features by their predictive redundancy (not just input correlation).

```
clustering = shap.utils.hclust(X_train, y_train)

# A built-in bar plot that dendrograms correlated features together

explanation = shap.Explanation(

    values=sv,

    base_values=np.full(len(X_test), expected_value),

    data=X_test.values,

    feature_names=list(X_test.columns),

)

shap.plots.bar(explanation, clustering=clustering, clustering_cutoff=0.5, show=True)
```

How to read it: features whose branches merge *below* the cutoff line are combined visually into a single bar — SHAP is telling you they are mutually redundant at that threshold. Use this as a candidate group definition, then confirm with domain knowledge before locking it in. `clustering_cutoff` is a hyperparameter: 0.5 is a reasonable starting point, lower values produce more (smaller) groups.

Important: `hclust` uses a predictive-redundancy metric — two features cluster together if one can predict the other given the target, not merely if they correlate. This makes it more robust than grouping by raw Pearson correlation, which can miss non-linear substitutes.

10. Validator's checklist

Before signing off on an interpretability artefact that uses Group SHAP, the second-line validator should verify each of these:

1. **Group definitions are documented and versioned** with the model artefact. Not reconstructable from plots after the fact.
2. **No feature appears in two groups.** Grouping is a partition.
3. **Raw SHAP values are summed, not absolute values.** Check the aggregation function in code, not just the plot.
4. **Reconstruction identity holds**: `sum(group_SHAP) + expected_value == model.raw_score` for all rows, to numerical tolerance. Automated assertion in the pipeline.
5. **Stability across retrains**: rank correlation of group importances across N bootstrap refits ≥ 0.9. Feature-level version typically 0.4–0.6 on correlated data; the gap is the point.
6. **Intra-group SHAP correlation is reported**, not just feature correlation. It is the empirical justification for the grouping choice.
7. **Background dataset is documented**: size, whether it's train or a representative sample, and the `feature_perturbation` setting of the explainer (`interventional` vs `tree_path_dependent`). These choices change SHAP values materially and must be frozen in the model card.
8. **Single-member groups are included in the group aggregation** so the total reconstructs. Don't silently drop them.

11. Common pitfalls

- **Summing absolute SHAP values for group importance.** Cancels legitimate net effects and is not a Shapley value. Use `mean(|group_SHAP|)` *after* the sum, not `sum(|SHAP|)` inside each group.
- **Defining groups after seeing the SHAP plot.** Circular and indefensible in a validation review.

- **Mixing explainer backgrounds.** Train-set background and test-set background produce different expected values and different SHAP distributions. Pick one and document it.
- **Applying Group SHAP to fix a genuinely collinear regression problem.** Group SHAP repairs *interpretation*; it does not repair the underlying modelling pathology. If your features are so redundant that grouping them leaves you with no new signal, you have a feature-engineering problem, not an explainability one.
- **Over-grouping.** If every group contains four-plus features, you've lost resolution. Groups should be tight: features that a domain expert would describe as "different measurements of the same thing".

12. References

- Kübler, R. *You Are Underutilizing SHAP Values — Feature Groups and Correlations*, Towards Data Science. Primary practical reference. Link
- Aas, K., Jullum, M., Løland, A. (2021). *Explaining Individual Predictions When Features Are Dependent: More Accurate Approximations to Shapley Values*. Artificial Intelligence 298. arXiv — theoretical foundation for conditional-expectation SHAP under dependence.
- Molnar, C. *Interpretable Machine Learning*, Chapter 18: SHAP. Canonical text; explicitly notes that a Shapley "player" can be a coalition of features.
- Basu, I., Maji, S. (2020). *Multicollinearity Correction and Combined Feature Effect in Shapley Values*. arXiv — adjustment-factor approach, useful for validation comparison.
- SHAP documentation — `shap.utils.hclust`, `shap.plots.bar` with `clustering` argument.

Document version: 1.0. Intended for internal team reference and code-review onboarding.